Static Analysis for Python

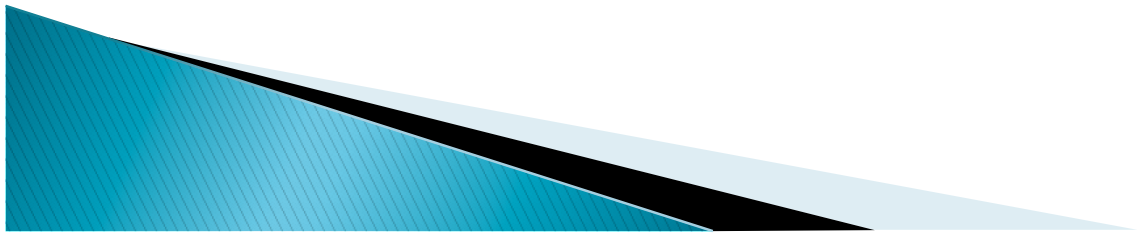
Guido van Rossum
guido@python.org

9th LASER summer school, Sept. 2012



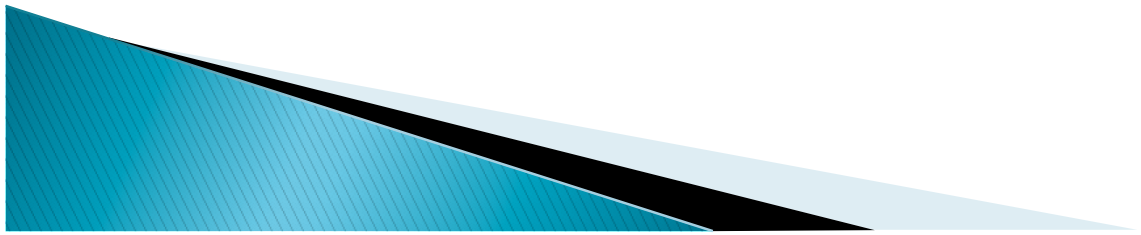
This Is All Speculation!

- Static analysis for Python is *hard*
- There are no good current tools
- There are a number of lint-like tools
- Some IDEs have some refactoring help
- Some projects exist that analyze a subset
  - e.g. PySonar by Yin Wang
- But wouldn't it be great to fix this?!



Possible Uses

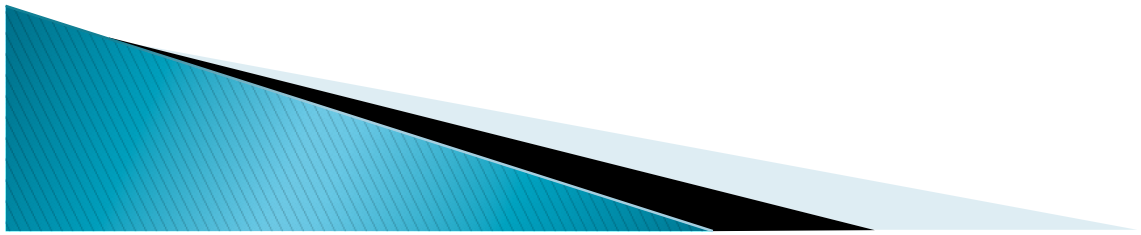
- IDE help: completion, refactoring
- Earlier detection of bugs ("lint")
- Automatic conversion to Python 3
- Optimizations???



Why Is It So Hard?

- The compiler is intentionally dumb
- There are no hints in the code about types
- Extremely dynamic language features
- Users like to write crazy hacks

- Let's go over some crazy hacks

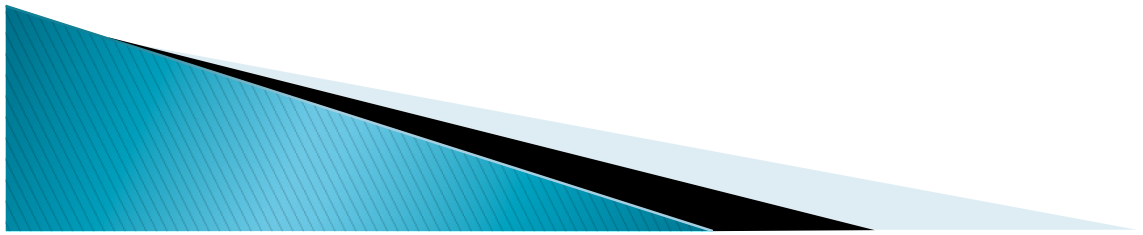


Manipulating the Import Path

```
def import_mod_from_dir(name, directory):
    save = sys.path[:]
    sys.path.insert(0, directory)
    module = __import__(name)
    sys.path[:] = save
    return module
```

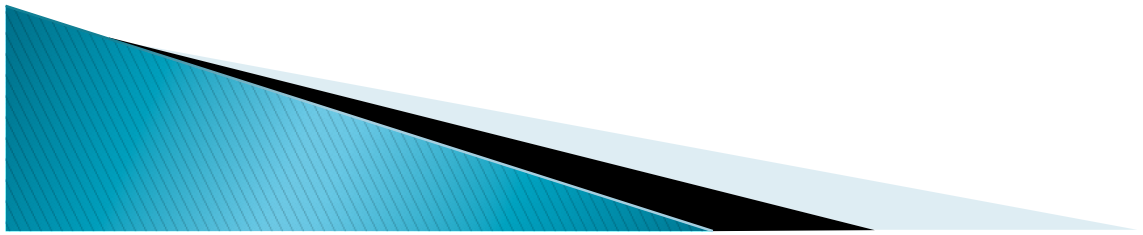
- `foo1 = import_mod_from_dir('foo', '/path/to/a')`
- `foo2 = import_mod_from_dir('foo', '/path/to/b')`
- Surprise! `foo1` and `foo2` are the same: `/path/to/a/foo.py`

- Even simpler:
  - `sys.path.insert(0, '/some/path')`
  - `import foo  # What does this import?`



Dynamic Dispatch

```
class Dynamic:
  def do_foo(self): ...; return 42
  def do_bar(self): ...; return 'abc'
  def do(self, name):
    method = getattr(self, 'do_' + name)
    return method()
d = Dynamic()
name = input()
x = d.do(name)  # What is the type of x?
```



Good Old Polymorphism

```
def sum(*xs):
  it = iter(xs)
  r = next(it)
  for x in it:
    r += x
  return r
```

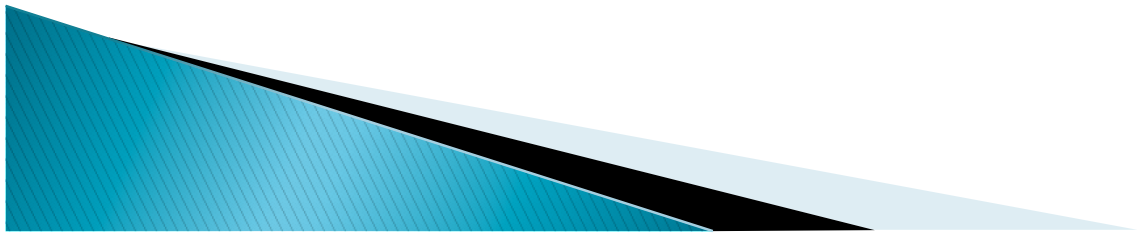
```
sum('a', 'b', 'c') == 'abc'
```

```
sum(1, 2, 3) == 6
```

```
sum(['foo'], ['bar']) == ['foo', 'bar']
```

```
x = sum(y)  # Obscure way to spell "x = y"
```

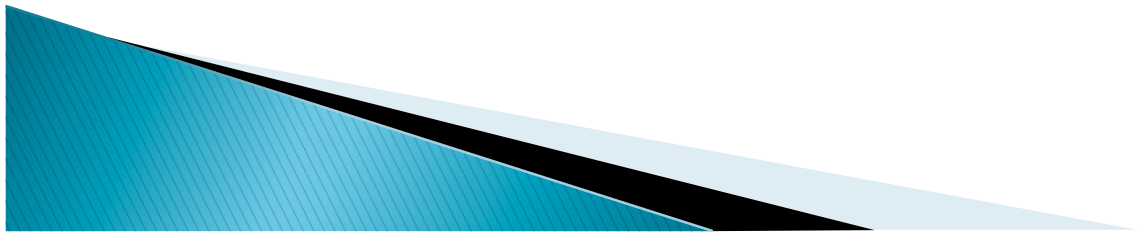
```
sum()  # Raises an exception
```



Operator Overloading

- ```
  class C(int):
    def __add__(self, other): return 'booh'
    def __radd__(self, other): return 'bah'
  ```
- `2 + C(2)  # prints 'bah'`
- `C(2) + 2  # prints 'booh'`

- Unlike Haskell, observing "x = y + z" does not tell is anything about the types of x, y, z

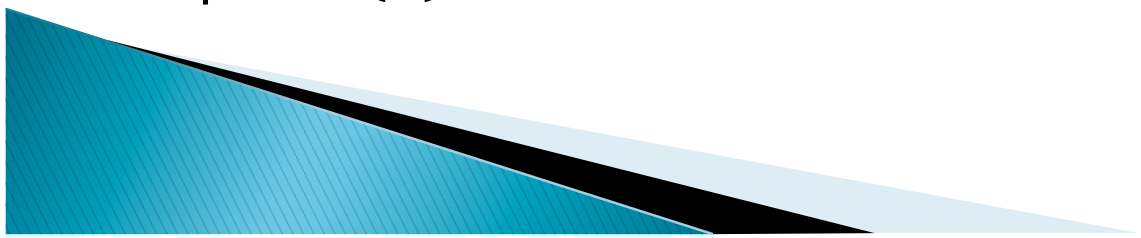

  ```

Playing With Locals (Or Globals)

```
x = 42
locals().setattr('x', 'abc')
print(x)  # ???
```

- The answer depends on whether we're inside a function or not. However:

```
global x
x = 42
globals().setattr('x', 'abc')
print(x)  # 'abc'
```

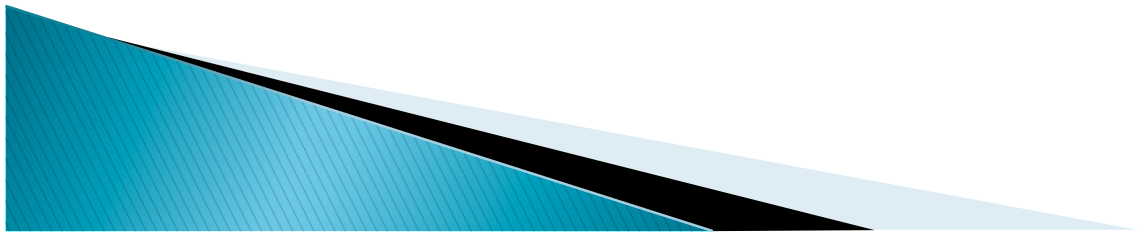


Modifying a Class

- ```
  class C:
    def foo(self): return 42
  ```
- `C.foo = lambda x: 'abc'`
- `C().foo()  # prints 'abc'`

- Of course it's quite possible that the assignment to C.foo is hidden elsewhere

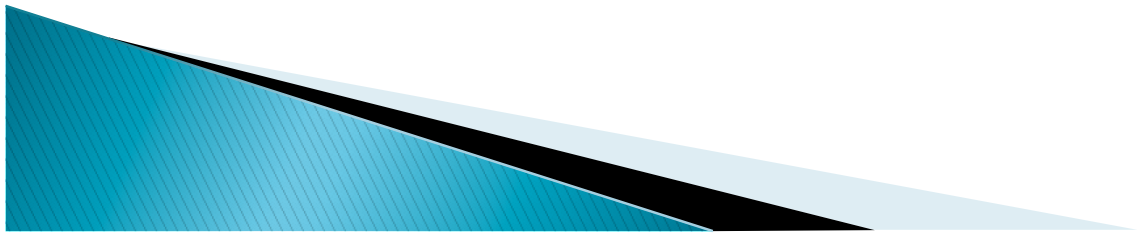
- Also known as "Monkey patching"
  - Sometimes used to fix broken code
  - Sometimes used to add new behavior


  ```

Dynamic Set Attribute

- Like previous example, but obfuscated:

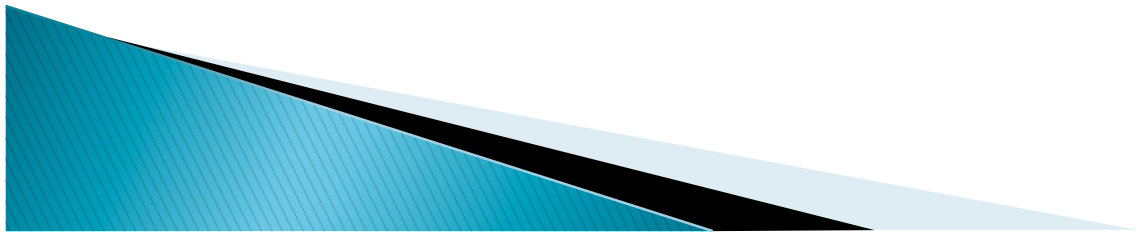
- `p = C`
- `q = 'oof'[::-1]`
- `setattr(p, q, lambda x: 'abc')`



Monkey Patching Builtins

- ```
  # foo.py:
  def foobar(x): return len(x) + 1
  ```
- ```
  # baz.py:
  import foo
  foo.len = lambda x: 42
  ```
- ```
  # bar.py:
  import foo
  foo.foobar('abc')  # 4 or 43 ???
  ```

- Why would anyone do this?
  - Unit tests!

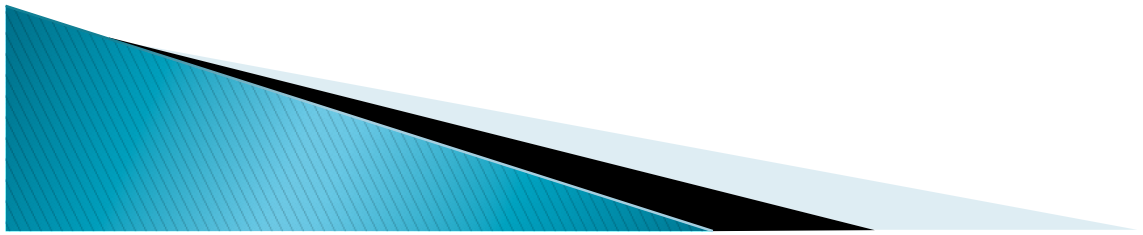

  ```

You Get The Picture

- Most lookups use a dynamic search path
- Most namespaces can be mutated
- No wonder Python is slow! :-)

- Exceptions:
  - local variable references are safe
  - so are "nonlocal" variables (closures)

- However, global vs. builtin is totally dynamic!



Scope Analysis

```
g1 = 42
def outer():
    o1 = 'a'
    def inner():
        x1 = 0
        print(x1)  # local
        print(o1)  # from outer()
        print(g1)  # global
        print(len)  # builtin
        print(z1)  # not defined
```

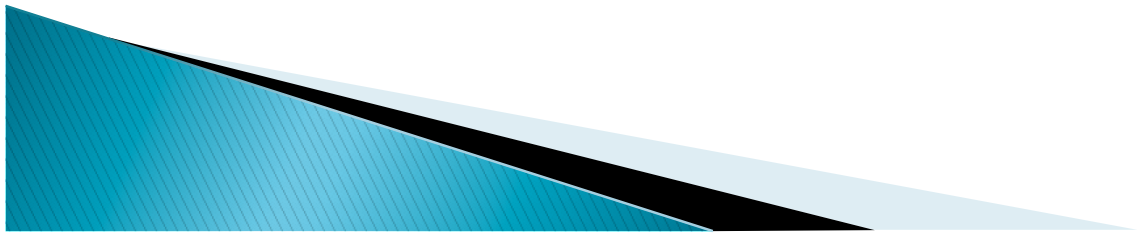
- The compiler knows the scopes of x1 and o1, but not of g1, len, and z1!
  - But an analysis tool should...



Heuristic: Possible Classes

```
class A:
  def foo(self): ...
class B:
  def bar(self): ...
def baz(x): x.foo()
```

- Assume x may be an A but not a B
- Can be fooled by monkey patching
  - But would be very rare



Heuristic: Import Path

- # foo.py:
  import bar
  x = bar.C()
- # bar.py:
  class C:
    ...

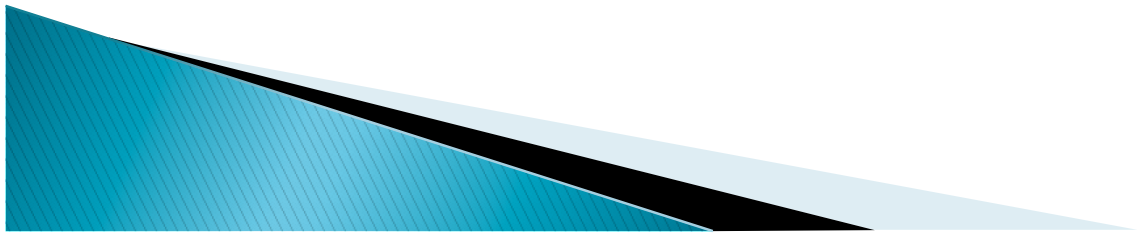
- Assume foo imports *this* bar.py
- Assume x is an instance of *this* class C
- Ignore sys.path shenanigans



How To Guess Argument Types

```
class C:
  def foo(self, arg):
    return arg.bar()
  def bar(self): return 42
class D:
  def bar(self): return 'abc'
```

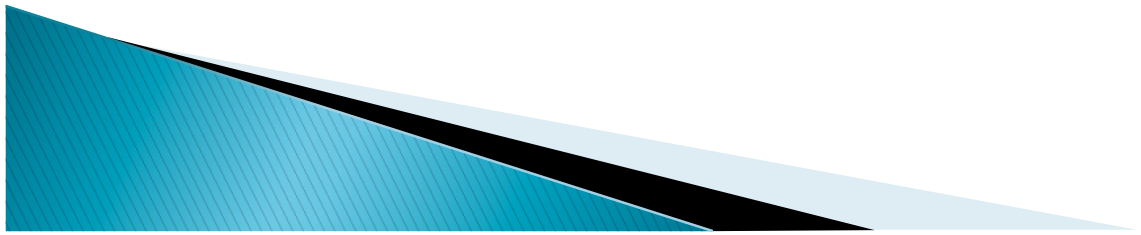
- What is the return type of C().foo(x)?
  - Depends on the type of x (duh)
  - Requires finding all call sites
  - Analysis may have to iterate back and forth



Conditionals

```
def foo(a, b):
  if a > 0:
    return a
  else:
    return b
q = foo(x, None)
```

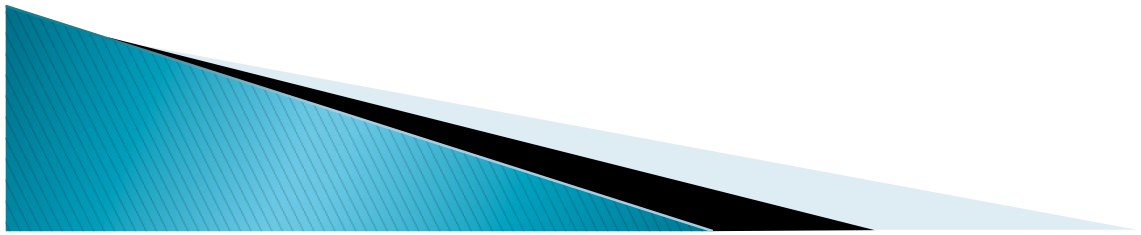
- What is the type of q?
  - Unify type of a and type of b, unless we know a > 0



Loops, Recursion

```
def foo(xs):
  a = None
  for x in xs:
    a = bar(x, a)
  return a
```

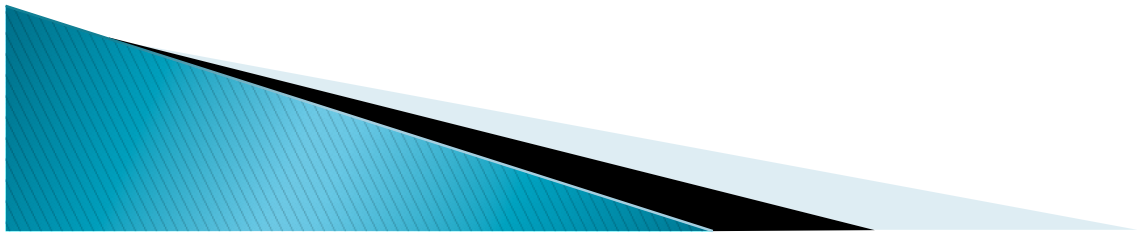
- How to compute the final type for a?
  - Worst-case scenario: bar(x, a) returns an N-dimensional list where N is 1+dim(a)
  - Needs fixed-point computation :-(



Attributes

- ```
  class C:
    a = 42
    def foo(self, a): self.a = a
  ```
- `c = C()`
- `bar(c)  # may or may not call c.foo('abc')`
- `a = c.a`

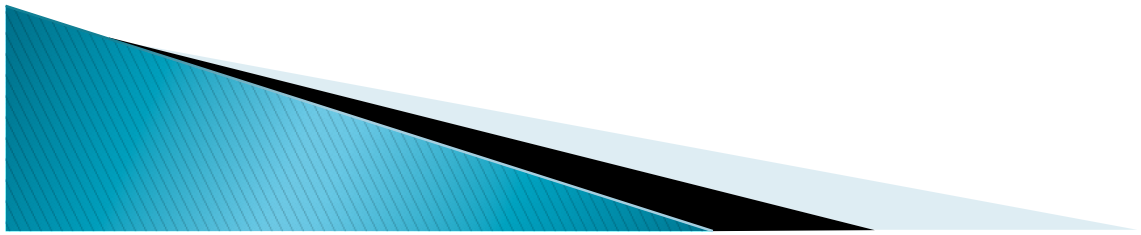
- What is the type of a?
  - Depends on what bar(c) did


  ```

Containers

- `a = {}`
- `a[key] = value  # For lots of keys and values`
- ```
  def foo(key):
    return bar(a[key])
  ```

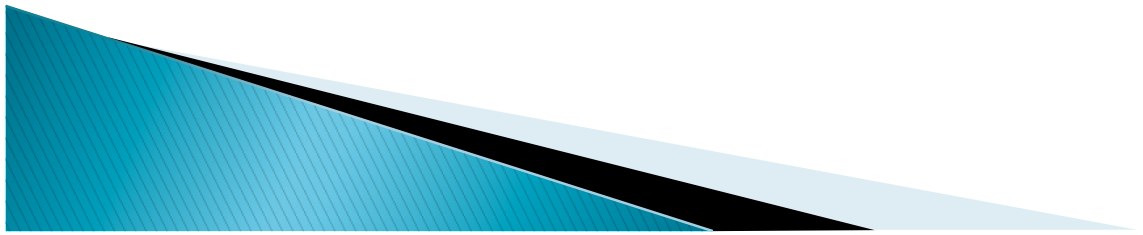
- What is the type of foo(x)?
  - Need to track down all assignments to a[key]


  ```

Containers, Attributes, Random

```
class C:
  def __init__(self, x):
    self.x = x
t = [C(42), C('abc')]
i = random.choice(range(len(t)))
x = t[i].x
```

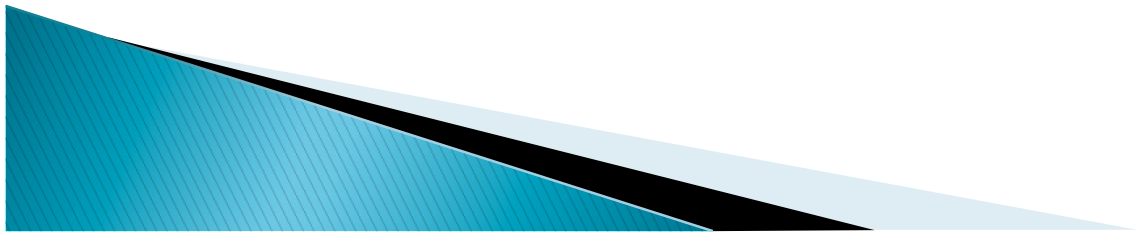
- What is the type of x?
  - Obviously int or str
  - Is the analysis smart enough to see that?
  - Can it tell that t[i] won't raise IndexError?



Aliasing

- `a = [.....]`
- `a[i] = 42`
- `a[j] = 'abc'`
- `x = a[i]  # 42 or 'abc' ???`

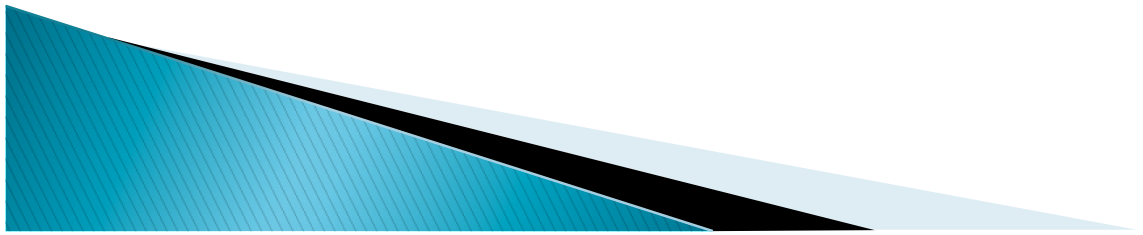
- `a.foo = 1`
- `a.__dict__[s] = ''`
- `setattr(a, n, None)`
- `x = a.foo  # 1, '' or None ???`



More Aliasing

- ```
  a = [1, 2, 3]
  ```
- ```
  def foo(b):
      b.append(42)
      return a
  ```
- ```
  foo(a)
  ```

- The possibilities are endless...


  ```

Binary Operators

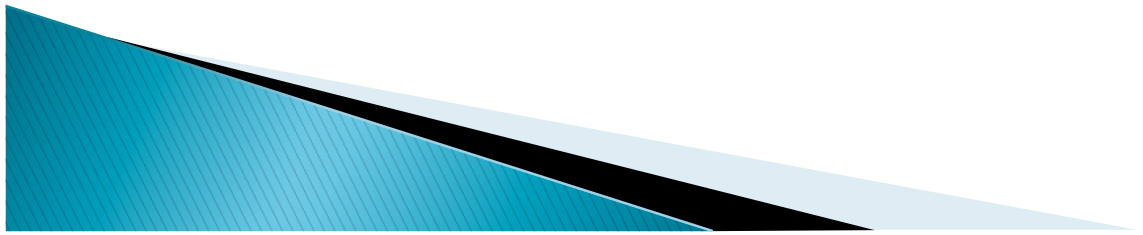
- "a + b" is equivalent to __add__(a, b):
  - ```
    def __add__(a, b):
        r = NotImplemented
        if hasmethod(a, '__add__'):
          r = a.__add__(b)
        if r is NotImplemented:
          if hasmethod(b, '__radd__'):
            r = b.__add__(a)
        if r is NotImplemented:
          raise TypeError
        return r
    ```
- (Except if type(b) is a subclass of type(a))
- `def hasmethod(x, name): name in dir(type(x))`


    ```

Exceptions

```
def foo(x, y):
    a = b = None
    try:
        a = bar(x)  # Can it raise MyException?
        b = baz(y)  # Ditto
    except MyException:
        pass
    return a, b
```

- What are the types of (a, b) returned?
  - What do we need to know about x, y, bar(), baz()?

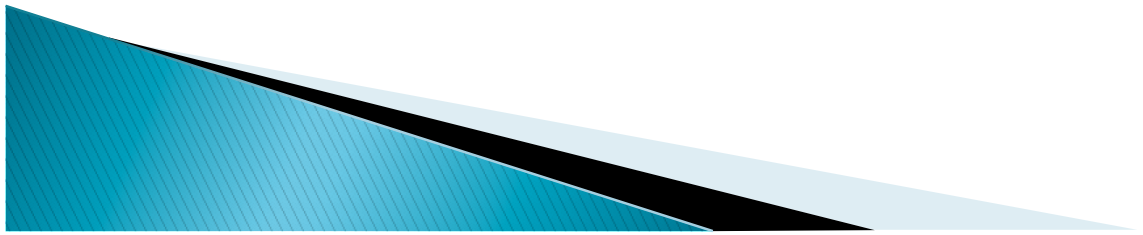


Threads

```
# Thread 1:
x = 42
print(x+1)
```

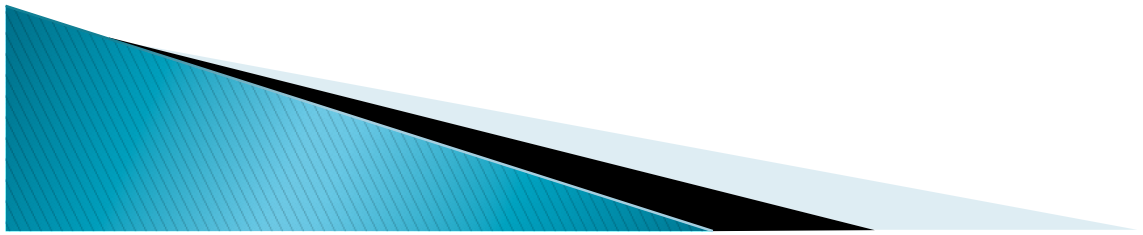
```
# Thread 2:
x = 'abc'
```

- Will print(x+1) fail with a TypeError?
  - Consider all possible interleavings of threads :-(
  - Lots of hard issues
  - Let's leave this for another time...



Heuristics Used By Humans

- def foo(func, x): ...
  - assume func is a function?
- def foo(filename): ...
  - assume filename is a string?
- def foo(x=1): ...
  - assume x is always an int? (or at least a number?)
- def foo(xs): ...
  def bar(xs): ...
  - assume xs in foo() and bar() have the same type?
- Etc.



Can We Learn From Others?

- Other languages (e.g. Haskell) have type inferencing systems; what can we reuse?
- Hindley-Milner might work, with suitable allowances made for Python's semantics
- Not all practical types expressible; e.g. (IIUC):
  - ```
    def foo(x, n):
      if n <= 0: return x
      else: return foo([x], n-1)
    ```
  - `foo(1, 1) == [1]`
  - `foo(1, 3) == [[[1]]]`
- "computing typings is a hopeless venture in general" (Wikipedia article on HM)
    ```

Please Help!

- My head just exploded…
- How to eat this elephant?
- Parsing and deciding scopes is easy enough
- Can we ever hope to determine the type of

```
def foo(x):
  return x + y
```

  without knowing all its call sites?
- Can we hope to track down all its call sites?
- What about unification?
- Does it matter?

